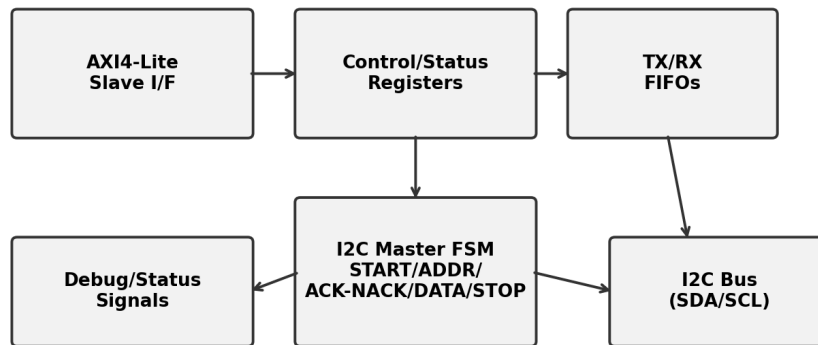


WindBorne Systems — Firmware Application Questions

Ethan Maynard Pinto | ethanpinto1234@gmail.com | 682-551-2903

1. Show us a cool thing you've built

Custom I2C Master Controller — Verilog / AXI4-Lite / FIFO



This is a self-directed project, not tied to a specific course: a custom I2C master controller written in Verilog that sits behind an AXI4-Lite interface with FIFO-buffered TX/RX paths. What makes it fun is that it's a full hardware/software boundary in miniature — the FSM has to get START/repeated-START, multi-byte transfers, and ACK/NACK handling exactly right at the bit level, while the AXI4-Lite side has to expose all of that cleanly as memory-mapped registers a CPU can actually poll or interrupt on. Getting both halves to agree on timing (and debugging it with nothing but waveforms and register dumps) is the part I'm most proud of.

2. Describe your setup

Day to day I live in VS Code — it's my default for C, Python, and general scripting because the extension ecosystem (PlatformIO-style workflows, Git integration, remote SSH) makes it flexible enough to not fight me. For FPGA work I'm in Vivado, mostly because that's what the toolchain requires for synthesis, implementation, and the ILA for waveform debugging — I don't fight that one either, I just live with the click-through-a-GUI parts of it. For ARM Cortex-M work (STM32, TM4C123) I use Keil uVision, since it's got the tightest debugger integration for stepping through interrupt-heavy code and watching registers in real time. Over time I've moved away from trying to force everything into one editor and toward just using whatever tool has the best debugger for the silicon in front of me — Vivado and Keil for the hardware-specific debugging, VS Code for everything else.

3. A PCB resets randomly once every few hours

First thing I'd check is power: brown-out detection, decoupling caps, and whether the reset lines up with a load spike (motor kicking on, a relay, anything inductive). I'd put a scope on the reset pin and Vcc simultaneously and just wait — if the reset is preceded by a dip on Vcc, that's a power problem, not a

firmware problem, and I'd stop looking at code entirely. If power looks clean, next is the watchdog: is it actually a reset, or a WDT timeout from something hanging? I'd check the reset-cause register the MCU almost always provides (POR vs. WDT vs. external vs. brown-out) before writing a single line of debug code, since that alone usually narrows it to one of three categories. From there I'd suspect ESD/EMI if it correlates with a nearby switching event, or a stack/heap overflow if it correlates with a specific code path running long. I'd change strategy once I can reliably correlate the reset with something (time of day, load, temperature) rather than treating it as random — at that point it's not a "debug everything" problem anymore, it's a "debug this one variable" problem. On my HX711 load cell project, an intermittent bad-looking reading turned out to be a sign-extension bug in how I was unpacking the 24-bit two's complement value, not a hardware fault at all — a good reminder that a "random" symptom is often a very deterministic bug that just doesn't trigger under the same conditions every time.

4. What do you think firmware/embedded engineers do wrong?

I think there's an over-reliance on "just add a delay" as a substitute for actually understanding the timing or race condition underneath — it makes the symptom go away without fixing the cause, and it comes back the moment the clock speed, load, or silicon batch changes. I also think RTOS adoption happens earlier than it needs to; a lot of projects would be simpler, more debuggable, and more power-efficient as a well-structured bare-metal superloop with interrupts, and only actually need a scheduler once there's real concurrency to manage. Related to that, I think power budgeting gets treated as a final polish step instead of a first-class design constraint, which is backwards for anything battery-powered or thermally constrained — by the time you're optimizing power at the end, the architecture is already locked in around assumptions that don't hold. Last one: I think a lot of engineers under-trust the reset-cause register and other free diagnostic data the MCU is already giving them, and jump straight to adding print statements or breakpoints instead of reading what the chip already told them.

5. A sketchy firmware fix that kept the project alive

While debugging an I2C master FSM on a Zynq-7000 targeting an MCP23008 GPIO expander, I found a `busy_reg` race condition where the busy flag could get sampled one cycle before it actually settled, which showed up as occasional dropped or duplicated transactions. The correct fix was restructuring the FSM so ACK sampling had its own dedicated state instead of being tacked onto the tail of the data-phase state — but that meant re-verifying the whole transaction sequence, and I had a demo the next day. So the sketchy interim fix was inserting an extra one-cycle wait state before the `busy_reg` check, which papered over the race without actually solving it. It worked — the demo went fine — but I knew it was fragile because it was timing-dependent rather than structurally correct, so I went back afterward and did the real fix: a proper dedicated ACK state with correctly aligned sampling. I'm not proud of shipping the band-aid, but I am a little proud that I **knew** exactly why it was fragile and fixed it for the right reason afterward instead of just leaving it alone because "it worked."